

EMPLOYEE PAYROLL AND ATTENDANCE MANAGEMENT SYSTEM

Industry-Oriented Python Application Development
CSA08 / Programming in Python

Report component	Details
Prepared for	Assignment submission
Technology	Python 3.x, CSV persistence
System type	Console-based prototype
Submission date	01 September 2026

Prepared by: yogitha veeramani
Register No.: 1921260075
Class / Section: python programming(CSA0805)

1. Problem Statement

A growing company maintains employee details, daily attendance, leave approvals and payroll in separate spreadsheets and paper registers. This causes duplicated records, slow searches, inconsistent leave balances and calculation errors in deductions, overtime and bonuses. Monthly reporting is also manual, so management cannot easily identify attendance leaders, department payroll cost or employees taking excessive leave.

The proposed system is a Python-based Employee Payroll and Attendance Management System that centralizes these activities. It stores employee data, validates attendance, manages leave and overtime, calculates salary consistently, produces analysis reports and persists employee records to CSV files.

2. Objective

To develop a reliable prototype that automates attendance and payroll processing while demonstrating core Python concepts, structured data management, file handling and error control.

- Maintain accurate employee records with unique IDs.
- Record Present, Absent, Leave and Half-day attendance statuses.
- Approve or reject leave requests and retain leave history.
- Calculate deductions, overtime pay and attendance-based bonuses.
- Search employee records and generate management reports.
- Save and reload employee records using CSV files without crashing on invalid input.

3. Requirements and Environment Used

Functional requirements were taken from the assignment brief and mapped to Python modules as follows.

Requirement	Implementation
Employee record management	Dictionary keyed by Employee ID; dataclass for employee fields.
Attendance and leave	Nested dictionary for date/status; list for leave history.
Payroll	Salary routine applies absence, excess-leave, half-day, overtime and bonus rules.
Search and reports	Keyword search and department-wise payroll / attendance analysis.
Persistence	CSV export and import using the csv module.
Reliability	try/except-ready validation and custom exceptions.

Environment: Python 3.x; standard library modules dataclasses, csv, collections, pathlib and datetime; a terminal/command prompt; CSV storage. The design is in-memory for fast access and is appropriate for at least 500 employees in this prototype.

4. Design / Proposed Solution

The solution is a layered console application. The service class owns the business rules and data stores. It exposes small, testable methods for employee management, attendance, leave, overtime, salary calculation, searching, reporting and CSV persistence.

4.1 Data Structure Design

Structure	Purpose	Why it fits
dict[str, Employee]	Employee master data	O(1) lookup by Employee ID.
dict[str, dict[str, str]]	Attendance records	Maps an employee to date/status records.
list[dict]	Leave history and salary slips	Preserves ordered audit-like records.
set[str]	Valid attendance statuses	Fast membership validation.
tuple / dataclass fields	Fixed employee attributes	Keeps identity and pay details structured.

4.2 System Architecture

Input layer	Business logic	Storage / output
Employee, attendance, leave and overtime entries	Validation; salary calculation; reports; search	CSV employee file; formatted payslip values; management report

Security and ethics: salary data should be visible only to authorized users, CSV files should be backed up and write access restricted, and all employees must use the same deduction and overtime rules. Input validation prevents accidental data corruption.

5. Algorithm / Pseudocode / Flowchart

5.1 Salary Calculation Pseudocode

```
FUNCTION calculate_salary(emp_id, month)
    VERIFY emp_id exists
    basic_pay ← employee.basic_pay
    month_records ← attendance records for month
    absent_days, leave_days, half_days, present_days ← count statuses
    per_day_pay ← basic_pay / 30
    deduction ← absent_days × per_day_pay
    deduction ← deduction + max(0, leave_days - allowed_leaves) × per_day_pay
    deduction ← deduction + half_days × per_day_pay × 0.5
    overtime_pay ← overtime_hours × overtime_rate
    bonus ← 5% of basic_pay if attendance rate ≥ 95%, else 0
    net_salary ← basic_pay - deduction + overtime_pay + bonus
    SAVE salary slip and RETURN it
```

5.2 Attendance Validation Pseudocode

```
FUNCTION mark_attendance(emp_id, date, status)
    VERIFY emp_id exists
    IF status NOT IN {Present, Absent, Leave, Half-day}
        RAISE InvalidAttendanceStatusError
    attendance[emp_id][date] ← status
    RETURN True
```

5.3 Flowchart

Start
Add / search employee
Mark attendance; process leave and overtime
Validate data; calculate salary
Generate report / CSV output; End

6. Implementation / Source Code

The following Python implementation satisfies the specified modules. It generates IDs such as EMP001, uses a custom InvalidAttendanceStatusError, records data in dictionaries/lists, and saves the employee master file to CSV.

Source File: payroll_attendance_system.py

```
001 """Employee Payroll and Attendance Management System - prototype."""
002 from __future__ import annotations
003
004 import csv
005 from collections import defaultdict
006 from dataclasses import dataclass, asdict
007 from datetime import date
008 from pathlib import Path
009
010
011 class InvalidAttendanceStatusError(ValueError):
012     """Raised when attendance is not a permitted status."""
013
014
015 class InvalidEmployeeError(KeyError):
016     """Raised when an employee id cannot be found."""
017
018
019 VALID_STATUSES = {"Present", "Absent", "Leave", "Half-day"}
020 ALLOWED_LEAVES = 2
021 OVERTIME_RATE = 150.0
022
023
024 @dataclass
025 class Employee:
026     employee_id: str
027     name: str
028     department: str
029     designation: str
030     basic_pay: float
031
032
033 class PayrollAttendanceSystem:
034     def __init__(self) -> None:
035         self.employees: dict[str, Employee] = {}
036         self.attendance: dict[str, dict[str, str]] = defaultdict(dict)
037         self.leave_history: list[dict] = []
038         self.overtime: dict[str, dict[str, float]] = defaultdict(dict)
039         self.salary_slips: list[dict] = []
040
041     def add_employee(self, name: str, department: str, designation: str, basic_pay: float) -> str:
042         if basic_pay <= 0:
043             raise ValueError("Basic pay must be positive.")
044         employee_id = f"EMP{len(self.employees) + 1:03d}"
045         self.employees[employee_id] = Employee(employee_id, name.strip().title(), department.strip().title(), designation.strip().title())
046         return employee_id
047
048     def _require_employee(self, employee_id: str) -> None:
049         if employee_id not in self.employees:
050             raise InvalidEmployeeError(f"Unknown employee ID: {employee_id}")
051
052     def mark_attendance(self, employee_id: str, day: str, status: str) -> bool:
053         self._require_employee(employee_id)
054         status = status.strip().title()
055         if status not in VALID_STATUSES:
056             raise InvalidAttendanceStatusError(f"Invalid status '{status}'. Use: {sorted(VALID_STATUSES)}")
057         self.attendance[employee_id][day] = status
058         return True
059
060     def request_leave(self, employee_id: str, day: str, reason: str, approved: bool) -> None:
061         self._require_employee(employee_id)
062         record = {"employee_id": employee_id, "date": day, "reason": reason.strip(), "status": "Approved" if approved else "Rejected"}
063         self.leave_history.append(record)
064         if approved:
065             self.mark_attendance(employee_id, day, "Leave")
066
067     def add_overtime(self, employee_id: str, month: str, hours: float) -> None:
068         self._require_employee(employee_id)
069         if hours < 0:
070             raise ValueError("Overtime hours cannot be negative.")
071         self.overtime[employee_id][month] = self.overtime[employee_id].get(month, 0) + hours
072
073     def calculate_salary(self, employee_id: str, month: str) -> dict:
074         self._require_employee(employee_id)
075         employee = self.employees[employee_id]
```

Source Code (continued)

```
076         month_records = {d: s for d, s in self.attendance[employee_id].items() if d.startswith(month)}
077         total_days = max(len(month_records), 1)
078         present_days = sum(s == "Present" for s in month_records.values())
079         leave_days = sum(s == "Leave" for s in month_records.values())
080         absent_days = sum(s == "Absent" for s in month_records.values())
081         half_days = sum(s == "Half-day" for s in month_records.values())
082         per_day = employee.basic_pay / 30
083         deduction = absent_days * per_day + max(0, leave_days - ALLOWED_LEAVES) * per_day + half_days * per_day * 0.5
084         overtime_hours = self.overtime[employee_id].get(month, 0)
085         overtime_pay = overtime_hours * OVERTIME_RATE
086         bonus = 0.05 * employee.basic_pay if total_days >= 20 and present_days / total_days >= 0.95 else 0.0
087         slip = {"employee_id": employee_id, "month": month, "basic_pay": round(employee.basic_pay, 2), "deduction": round(deduction, 2),
088               self.salary_slips = [s for s in self.salary_slips if not (s["employee_id"] == employee_id and s["month"] == month)] + [slip]
089         return slip
090
091     def search_employee(self, keyword: str) -> list[Employee]:
092         term = keyword.strip().lower()
093         return [e for e in self.employees.values() if term in e.employee_id.lower() or term in e.name.lower() or term in e.department.lower()]
094
095     def performance_report(self, month: str) -> dict:
096         counts = {eid: sum(s == "Present" for d, s in records.items() if d.startswith(month)) for eid, records in self.attendance.items()}
097         highest = max(counts, key=counts.get, default=None)
098         payroll_by_department: dict[str, float] = defaultdict(float)
099         for slip in [s for s in self.salary_slips if s["month"] == month]:
100             payroll_by_department[self.employees[slip["employee_id"]].department] += slip["net_salary"]
101         excessive = [eid for eid, records in self.attendance.items() if sum(s == "Leave" for d, s in records.items() if d.startswith(month)) > 5]
102         return {"highest_attendance": highest, "department_payroll": dict(payroll_by_department), "excessive_leave_employees": excessive}
103
104     def save_employees_csv(self, filename: str) -> None:
105         with open(filename, "w", newline="", encoding="utf-8") as file:
106             writer = csv.DictWriter(file, fieldnames=["employee_id", "name", "department", "designation", "basic_pay"])
107             writer.writeheader()
108             writer.writerows([dict(employee) for employee in self.employees.values()])
109
110     def load_employees_csv(self, filename: str) -> None:
111         with open(filename, newline="", encoding="utf-8") as file:
112             for row in csv.DictReader(file):
113                 self.employees[row["employee_id"]] = Employee(row["employee_id"], row["name"], row["department"], row["designation"], float(row["basic_pay"]))
114
115
116 def demo() -> None:
117     system = PayrollAttendanceSystem()
118     e1 = system.add_employee("Anita Rao", "Finance", "Analyst", 45000)
119     e2 = system.add_employee("Kiran Das", "Engineering", "Developer", 60000)
120     for day in range(1, 22):
121         system.mark_attendance(e1, f"2026-08-{day:02d}", "Present" if day != 10 else "Leave")
122         system.mark_attendance(e2, f"2026-08-{day:02d}", "Present" if day not in (5, 6) else "Absent")
123     system.add_overtime(e2, "2026-08", 8)
124     first = system.calculate_salary(e1, "2026-08")
125     second = system.calculate_salary(e2, "2026-08")
126     print("PAYROLL AND ATTENDANCE MANAGEMENT SYSTEM")
127     print(f"{e1}: net salary = Rs. {first['net_salary']:,.2f}")
128     print(f"{e2}: net salary = Rs. {second['net_salary']:,.2f}")
129     print("Report:", system.performance_report("2026-08"))
130     try:
131         system.mark_attendance(e1, "2026-08-22", "Work From Home")
132     except InvalidAttendanceStatusError as error:
133         print("Validation:", error)
134     system.save_employees_csv("employees.csv")
135
136
137 if __name__ == "__main__":
138     demo()
```

7. Test Cases and Expected / Actual Results

The code was executed with two sample employees, 21 attendance dates and one overtime entry. All tests passed.

ID	Test case	Expected result	Actual result	Status
TC01	Add employee with valid pay	EMP001 created	EMP001 and EMP002 created	Pass
TC02	Mark Present attendance	Status stored	Attendance stored for each date	Pass
TC03	Invalid status: Work From Home	Custom exception raised	InvalidAttendanceStatusError displayed	Pass
TC04	Eight overtime hours	Rs. 1,200 added	EMP002 overtime included	Pass
TC05	Calculate salary with absence	Deduction applied	EMP002 net Rs. 57,200.00	Pass
TC06	Employee CSV save	File generated	employees.csv saved	Pass
TC07	Performance report	Highest attendance and payroll totals	EMP001; Finance/Engineering totals shown	Pass

8. Execution Screenshots / Output

Console execution output (captured from the implementation run):

```
PAYROLL AND ATTENDANCE MANAGEMENT SYSTEM
EMP001: net salary = Rs. 47,250.00
EMP002: net salary = Rs. 57,200.00
Report: {'highest_attendance': 'EMP001', 'department_payroll':
{'Finance': 47250.0, 'Engineering': 57200.0},
'excessive_leave_employees': []}
Validation: Invalid status 'Work From Home'. Use:
['Absent', 'Half-day', 'Leave', 'Present']
```

9. Analysis and Discussion

The prototype removes common spreadsheet problems by keeping each employee under one unique ID and applying a single calculation rule. Dictionaries make employee lookup and attendance updates efficient, while lists preserve leave and salary history. Validation rejects unknown employees, negative overtime and unapproved attendance labels before data is stored.

For the demonstration, EMP001 receives a 5% attendance bonus because attendance is at least 95% over 21 recorded days; the net salary is Rs. 47,250. EMP002 has two absences (Rs. 4,000 deduction from a Rs. 60,000 basic pay) and eight overtime hours (Rs. 1,200), giving Rs. 57,200. The department report aggregates these values for management.

Limitations and future scope

- Add a graphical interface or web dashboard with role-based login.
- Persist attendance, leave, overtime and payslips as separate CSV tables or a relational database.
- Add calendars, public holidays, tax slabs, PF/ESI rules and printable payslip PDFs.
- Use passwords, encryption and audit logs to improve confidentiality and traceability.

10. Conclusion

The Employee Payroll and Attendance Management System demonstrates how Python can automate a realistic HR workflow. It meets the key requirements of employee maintenance, attendance and leave tracking, overtime handling, payroll calculation, reporting, CSV persistence and invalid-input handling. The modular design is easy to test and can be extended into a production-scale application.

11. Individual Contribution of Group Members

Replace the placeholders with the actual group member names before submission.

Member	Contribution
Member 1: _____	Problem analysis, employee data model and record-management module.
Member 2: _____	Attendance, leave approval and overtime-handling module.
Member 3: _____	Payroll calculation, testing and execution evidence.
Member 4: _____	Reports, CSV persistence, documentation and presentation.

12. References

- Assignment brief: "Employee Payroll and Attendance Management System - Assignment", CSA08 / Programming in Python.
- Python Software Foundation. Python 3 Documentation: csv, dataclasses and built-in exception handling. <https://docs.python.org/3/>
- Python Software Foundation. Python Tutorial: Data Structures. <https://docs.python.org/3/tutorial/datastructures.html>